

Technische Präsentation

RO45 Robotik Projekt 3 – Automatische Sortieranlage

NimSort42

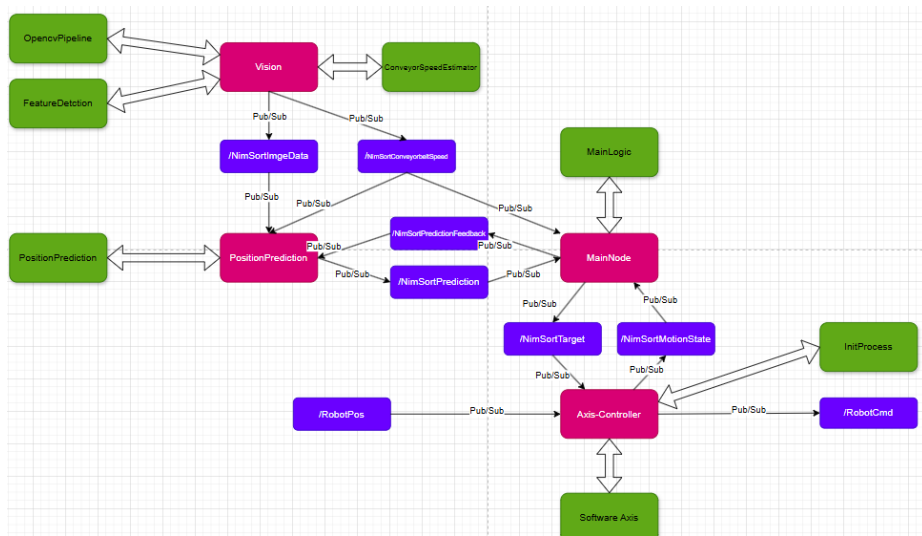
Hochschule Kempten

29. Juni 2026

Inhalt

- Softwarearchitektur
- Designentscheidungen
- Technische Details
- Anpassungen & Schwierigkeiten
- Lessons Learned
- Zusammenfassung

Softwarearchitektur



Sentinel

Designentscheidung:

Sentinel senden anstatt keine Nachricht, falls kein Objekt erkannt wurde. Bei keiner Nachricht kontrolliertes Abbrechen des Prozesses.

Vorteile:

- Prozessschritte können auf Sentinel reagieren
- Keine Abhängigkeit von der Reihenfolge der Nachrichten
- Einfaches Debugging

Bewertung:

- Funktioniert zuverlässig
- Simple Umsetzung

$[-1.0, -1.0, -1.0, -1]$

Datenhaltung in der PositionPrediction

Designentscheidung:

Objektdaten werden als Dataclass in einem Dictionary organisiert.

Vorteile:

- Singuläre Datenhaltung an einem Ort
- Speicherprozess aus der Vision/ ML auslagern

Bewertung:

- gute Nachvollziehbarkeit
- Parallele Entwicklung möglich

Pick Prozess

Designentscheidung:

Drive by Pick Prozess anstatt One-Shot Pick Prozess

Vorteile:

- Zuverlässiger als One-Shot Pick Prozess
- Leichter, da nur Positionsgesteuert

Bewertung:

- Größer Chance zu greifen
- Robuste Logik

Feedback von Main zur PositionPrediction

Designentscheidung:

Die PositionPredictionNode erhält zusätzlich Feedback von der MainNode.

Vorteile:

- Reduzierte Datenhaltung in der MainNode
- Weniger Datenübertragung

Bewertung:

- Komplexität gering
- Architektur bleibt konsistent
- Logische Erweiterung der PositionPrediction

PD-Regler vs. PDF-Regler

Designentscheidung:

Für die Regelung der Achsen wird ein PD-Regler verwendet, nicht wie ursprünglich geplant ein PDF-Regler

Warum PD-Regler?

- Einfache Parametrierung
- Stabiler Betrieb im Testaufbau
- Erweiterung auf PDF möglich

Warum kein PDF-Regler?

- Unsicherheiten mit dem gefundenen Multiplikator 0.8
- Zusätzlicher Aufwand

Fail-Safe statt Fail-Operational

Designentscheidung:

Verzicht auf Fail-Operational zugunsten eines definierten Safe-States mit anschließendem Neustart.

Begründung:

- Kritische Fehler entlang: (Vision → Prediction → Main → AxisController)
- Nach Fehler ist Neustart erforderlich
- Fail-Operational bringt wenig Vorteile aber höhere Komplexität
- Automatische Rückkehr in sichere Maschinenposition

Fehler erkannt → Safe-State → Neustart möglich

Keine Aruco Marker → Homographie

Designentscheidung:

Koordinatentransformation via Homographie statt Aruco Marker

Aruco Marker (verworfen)

- Pixel → Kamera → Welt via Marker-Pose
- Overhead bei ROS2 tf2

Homographie (gewählt)

- Direkte Pixel → Welt Transformation
- Kein Aruco Marker nötig
- Für Projekt ausreichend

Kamera-Ausrichtung mit digitalem Overlay

Designentscheidung:

Kameraposition wird per digitalem Overlay kontrolliert / justiert

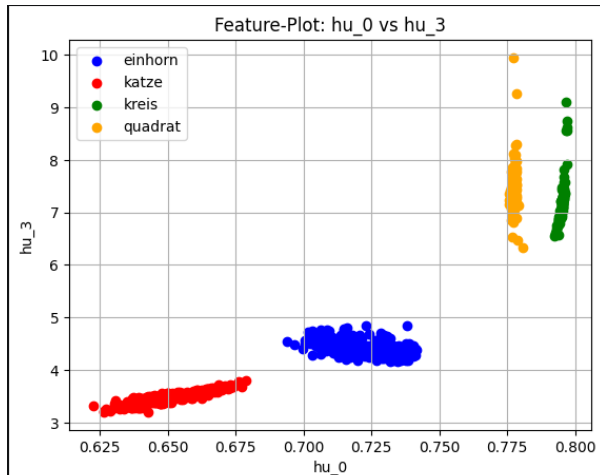
- Homographie erfordert reproduzierbare Kameraposition
- Overlay ermöglicht schnelle visuelle Kontrolle
- Reduziert Positionsfehler als Fehlerquelle

Decision Tree & Features

Hu-Momente: Invariant gegenüber Translation, Skalierung, Rotation und Spiegelung

Bereits zwei Merkmale reichen zur klaren Trennung:

Klasse	hu_0	hu_3
Katze	0,615 – 0,680	3,20 – 3,70
Einhorn	0,685 – 0,740	4,10 – 4,90
Quadrat	0,770 – 0,780	6,40 – 8,20
Kreis	0,780 – 0,800	6,60 – 9,10



Decision Tree & Features

Merkmalsraum:

hu_0 vs. hu_3 zeigt klar getrennte Cluster

- Klassen sind im Merkmalsraum differenzierbar
 - Trennungen verlaufen achsparallel
- ⇒ Einfache Schwellwertentscheidungen ($hu_0 < \theta$)
- ⇒ Decision Tree bildet genau diese Struktur ab

Komplexere Modelle (SVM, neuronales Netz) wären für klar separierbare Klassen unnötiger Rechenaufwand.

Homographie – Pixel zu Weltkoordinaten

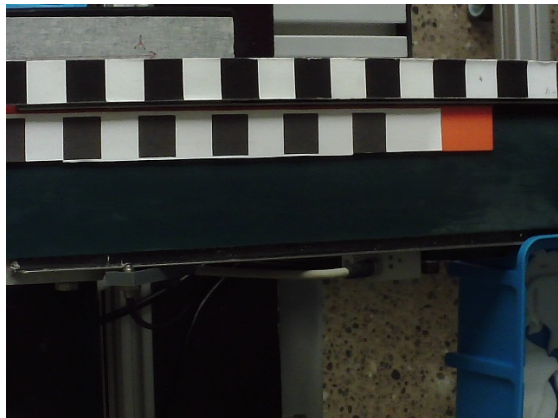
Ziel: Pixelkoordinaten $\rightarrow$ reale Weltkoordinaten (mm) via 3×3 -Matrix H

Kalibrierung:

- 5 Schachbrettquadrate, je 4 Eckpunkte
- 20 Punktepaare (Pixel $\leftrightarrow$ mm) manuell vermessen
- H berechnet via `cv2.findHomography()`

Laufzeit:

- Jede Pixelcoordinate wird mit H transformiert
- H bleibt konstant solange Kamera unverändert



Kommunikation Main und AxisController

Ziel

Eindeutige Kommunikation ohne Annahmen zu treffen.

Umsetzung

Die Kommunikation besteht aus zwei Messages.

NimSortTarget:

- `geometry_msgs/Point target_point`
- `int32 process_id`

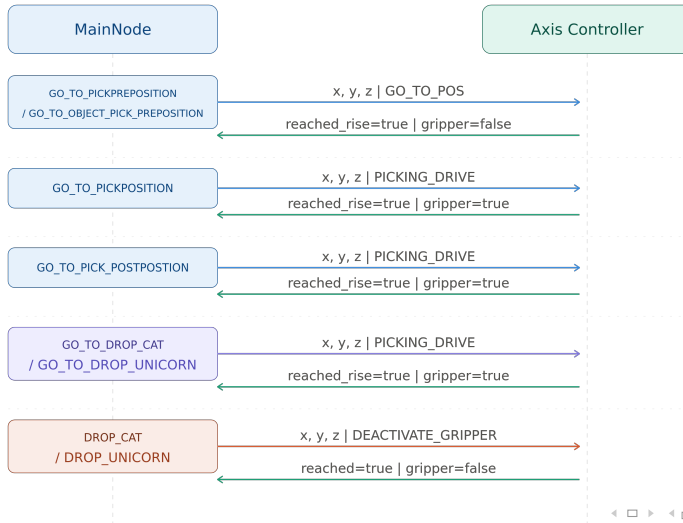
NimSortMotionState:

- `bool reached`
- `bool gripper_active`

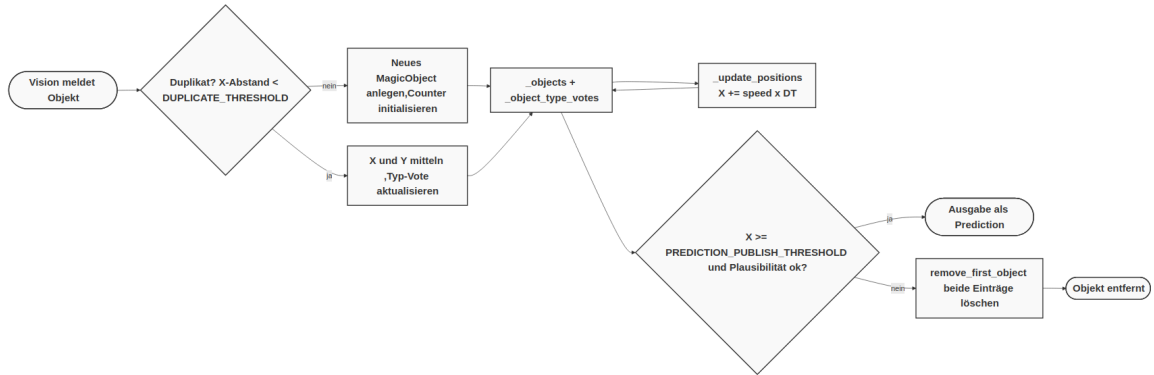
Process IDs

- 0 = Initialisierung Main Node
- 1 = Initialisierung Achsen
- 2 = Fahren zu einer Position
- 3 = Fahren mit speziellem reached
- 4 = Fahren mit aktiviertem Greifer
- 5 = Greifer deaktivieren

Pick Prozess



PositionPrediction Daten



PD-Regler

Regelprinzip

$$e = x_{soll} - x_{ist}$$

$$a = K_P \cdot e + K_D \cdot \dot{e}$$

$$\dot{e} = \frac{(e - e_{old})}{t_e - t_{e_{old}}}$$

- Ausgabe ist eine Beschleunigungsvorgabe

Ziel: Schnelle und stabile Annäherung an die Sollposition

Ablauf

- ➊ Position empfangen
- ➋ Fehler berechnen
- ➌ Fehleränderung bestimmen
- ➍ Beschleunigung berechnen

Förderbandgeschwindigkeit

Berechnung

$$v = \frac{\Delta x}{\Delta t}$$

- Keine direkte Messung
- Berechnung aus Objektpositionen
- Grundlage der Positionsvorhersage

Filterung

- Rohgeschwindigkeit ≥ 0
- Medianfilter
- Tiefpassfilter mittels PT1-Filter

Robuste Geschwindigkeitsschätzung trotz Messrauschen

Fail-Safe Konzept

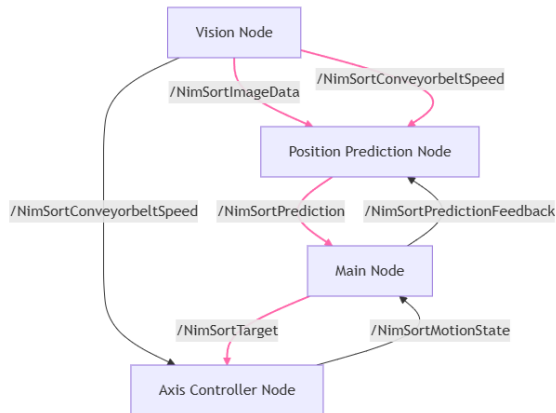
Grundidee: Safe-State durch Homing nach Fehler

Auslöser

- Fehler im AxisController
- Ausfall abhängiger Softwarekomponenten
- Fehlende robot_position-Daten

Vorteile

- Schutz von Mensch und Maschine
- Einfacher als Fail-Operational



Fehler → Homing → Safe-State

Anpassungen

- Topic Feedbacks
- Softwareaxis
- Koordinatensysteme
- Berechnung der Förderbandgeschwindigkeit

Schwierigkeiten

- Hardwareprobleme
- Kamerakalibrierung

Einheitliche Konzepte

Logging

- Schnellere Fehlersuche
- Einfachere Zuordnung zum Code
- Auffällige Abweichungen sichtbar

Interfaces

- Klare Schnittstellen
- Fehlende Funktionen früh erkannt
- Weniger Integrationsprobleme

Teamverständnis

- Gemeinsames Architekturverständnis
- Beiträge über Modulgrenzen hinweg
- Weniger Missverständnisse

Einheitliche Konzepte $\Rightarrow$ höhere Entwicklungsqualität

Fehlerhafte Hardware kostet Softwarezeit

Fehlersuche

- Hardwarefehler schwer erkennbar
- Suche nach nicht existierenden Softwarefehlern
- Verzögerte Ursachenanalyse

Robustheit

- Günstige Hardware erzeugt zusätzliche Fehlerfälle
- Höhere Anforderungen an die Software
- Mehr Absicherungen notwendig

Aufwand

- Hardwarereparaturen
- Zeitverlust außerhalb des Fachgebiets
- Weniger Entwicklungszeit

Hardwarequalität $\Rightarrow$ Entwicklungsaufwand

GitHub & Branches

Versionsverwaltung

- Branches für parallele Entwicklung
- Einfache und intuitive Merge-Strategie
- Änderungen isoliert entwickelbar
- Lauffähiger/getesteter Entwicklungsstand

Projektorganisation

- GitHub Projects
- Tracking von Anforderungen
- Transparenter Projektfortschritt
- Unterstützung der Zeitplanung
- Retrospektive möglich

Parallele Entwicklung → Kontrollierte Merges ⇒ Stabiler Entwicklungsstand

Plausibilitätschecks

Motivation

Fehlerhafte Koordinaten aus Bildverarbeitung oder Koordinatentrafo sind möglich.

Umsetzung

- Prüfung auf 3 Elemente
- X- und Y-Koordinaten werden gegen konfigurierbare Grenzwerte geprüft
- Bei Verstoß: Warnung und Verwerfung des Objekts

Lesson Learned

Der Realtime Plausibilitätscheck erkennt Fehler und Crashes werden verhindert. Auch während dem Entwicklungsprozess.

Fazit

- Geringer Implementierungsaufwand
- Hoher Sicherheitsgewinn

Meilensteine in Iterationen aufteilen

Vorgehen

Die Projektmeilensteine in Iterationen unterteilt. Iteration mit Ziel und Ergebnis

Vorteile

- Klare Struktur im Projektverlauf
- Frühzeitiges Erkennen von Problemen
- Simultane Aufgabenverteilung im Team

Lesson Learned

Große Meilensteine wirken übersichtlich, sind aber schwer nachzuvollziehen

Empfehlung

- Meilensteine bauen sich zusammen.
- Iterationen wurden in GitHub Project getrackt
- Regelmäßige Zwischenziele motivieren und halten den Überblick

Verarbeitungsgeschwindigkeit der Bildverarbeitung

Problem: Homographiematrix wurde für jedes Bild neu berechnet

- Kameraposition bleibt im Betrieb konstant: H ändert sich nie
- ⇒ Berechnung einmalig in `__init__()` ausgelagert
- ⇒ Konstante Ergebnisse nur einmalig berechnen

Belichtung

- Pipeline funktionierte zunächst korrekt aber nach kurzer Zeit keine Erkennung mehr
 - Ursache: veränderte Belichtung (z. B. Sonneneinstrahlung, Raumbeleuchtung, etc.)
- ⇒ Konsistente Belichtung ist neben der Kameraausrichtung der wichtigste Faktor für zuverlässige Objekterkennung

Zusammenfassung

Erreichte Ziele

- Themen aus der Vorlesung wurden umgesetzt
- Software- und Hardwareintegration erfolgreich durchgeführt
- Objekterkennung und Sortierung realisiert
- Modulare Architektur entwickelt
- Gute Teamarbeit :)



Naiv implementierte Mustererkennung sortiert Objekte random trotz 42 Verkabelungsproblemen.